

Deliverable 3: Integrated Technical Implementation Report

Technical Implementation Report: Radar-Based Artillery Firing Solution System

This document presents a comprehensive summary of the design, implementation, and analysis of the radar-guided artillery targeting system, integrating both the core simulation data management and the mathematical firing solution engine.

I. Executive Summary

The reported system is a C++ implementation of a radar-guided artillery targeting solution designed to automate the calculation of optimal firing angles. It efficiently processes radar-derived positional data into precise aiming instructions. The core innovation lies in using flexible expression trees to implement and manage the required mathematical formulas, providing a highly adaptable calculation framework. Data management relies on foundational C++ structures, including a First-In, First-Out (FIFO) Linked List for event logging and a Circular Linked List to model the continuous 360-degree radar sweep. This project serves as a practical demonstration of computer science concepts such as dynamic memory management, tree structures, and algorithmic optimization in a real-world simulation context.

II. System Architecture and Core Components

A. System Philosophy and Data Flow

The system adheres to a component-based architecture, ensuring each class has a single, well-defined responsibility. This modular design facilitates independent testing, maintenance, and potential reuse in various scenarios.

The high-level data flow involves Radar Input supplying target data to a Target Object (the threat representation), which is processed by the Gun Computer to yield the final Firing Solution.

B. Detailed Component Analysis

Component	Type	Primary Purpose	Key Features / Role
Vector2D	struct	Spatial Coordinate Management	Stores x and y coordinates; calculates distance and magnitude.
Target Class	class	Dynamic Threat Representation	Models moving aerial threats by tracking position and altitude. Calculates horizontal distance to the gun position.

Component	Type	Primary Purpose	Key Features / Role
Gun Class	class	Intelligent Aiming Computer	Manages a solutionHistory using a fixed-size pre-allocation (new FiringSolution[MAX_HISTORY]), requiring manual cleanup with delete[].
FiringSolution	struct	Comprehensive Aiming Package	Stores the calculated elevation 0 to 90 and azimuth 0 to 360 angles, horizontal distance, and a human-readable direction string.

III. Data Management and Simulation Structures

A. Detection Event Queue (FIFO Linked List)

This structure enforces a First-In, First-Out (FIFO) sequence, which is critical for the chronological processing of system alerts (Entry/Exit events).

- Implementation: A Singly Linked List built from QueueNode structs, maintaining queueFront for removal and queueRear for insertion.
- Complexity: Both enqueueEvent and dequeueEvent are highly efficient $O(1)$ (Constant Time) operations, achieved through a fixed number of pointer re-assignments. Cleanup using clearQueue is $O(Q_{\text{max}})$ (Linear Time) as it iterates through the maximum queue size.

B. Circular Sweep (Circular Linked List)

This structure models the continuous, rotational scanning motion of the radar antenna, providing a seamless 360-degree sweep by linking the last node back to the first.

- Implementation: The sweep is constructed with a fixed 16 nodes, representing 22.5 angular steps.
- Complexity: The initializeSweep() function is considered $O(1)$ (Constant Time) due to the fixed iteration count of 16. The advanceSweep() function, which simulates rotation, is the most efficient method for traversal, implemented as a single pointer assignment with $O(1)$ (Constant Time) complexity.

IV. Mathematical Calculation Engine

A. Expression Tree System

Expression Trees were chosen to represent mathematical formulas as hierarchical node structures, providing flexibility over hardcoded formulas. They can be dynamically built, modified, and evaluated.

- Node Hierarchy: The base class ExpressionNode is abstract, defining the virtual methods evaluate() and clone(). Subclasses, such as ConstantNode, inherit this structure.

- Purpose: Expression Trees are distinct from search-optimized structures (like BST or AVL trees) as their primary purpose is Formula evaluation based on Operator hierarchy.

B. Angle Computations

1. Elevation Angle: Calculation is based on the Inverse tangent relationship between altitude and horizontal range. The calculation includes validation logic to clamp the final elevation angle between 0 and 90.
2. Azimuth Angle: Calculation uses the Two-argument arctangent to determine the compass bearing. The logic includes specific handling for division by zero (when Delta x = 0) and normalization to the 0 to 360 range.

V. Performance Analysis and Optimization

A. Overall Time Complexity

Operation	Complexity	Justification
Targeting Calculations (Distance, Elevation, Azimuth, History Storage)	O(1)	Fixed arithmetic operations.
Expression Tree Evaluation	O(N)	N = number of tree nodes (typically small, < 10).
Core Detection Logic (Phase 3)	O(N × D)	Outer loop iterates through N global targets, with an inner linear search through D detected targets.
Worst-Case Detection (D ≈ N)	O(N^2)	Quadratic growth rate caused by the inefficient linear search used for checking existing detections in an array.

B. Key Optimizations Implemented

- Precomputation: Constants, such as 180.0/pi (rad to degrees), are precomputed to save run-time calculation.
- Inline Functions: Used for short, frequently called member functions (e.g., Vector2D::magnitude()) to minimize function call overhead.
- Reference Passing: Using const Target& target prevents costly object copies during function calls.

VI. Reliability and Validation

A. Test Case Design

Core test cases validate the calculateFiringSolution function under specific geometric scenarios:

- Test 1 (45°): Targets with equal height and horizontal distance.
- Test 2 (0°): Ground targets (altitude = 0.0).
- Test 3 (90°): Targets located directly above the gun (zero horizontal distance).

B. Failure Mode Analysis (FMEA)

Failure Scenario	System Response	Impact
Invalid target position	Returns default (0) solution.	Graceful degradation.
Memory allocation failure	Constructor throws exception.	Prevents system from entering an invalid state.
History buffer full	Overwrites the oldest entry.	Ensures bounded memory usage and no data loss.

VII. Extensibility and Future Enhancements

The modular design supports future enhancements, including:

- Moving Target Prediction: Incorporating velocity vectors and calculating a lead angle.
- Environmental Factors: Adding wind compensation and other ballistic corrections.
- Multi-Target Engagement: Implementing threat prioritization algorithms.